

Agente de Calidad de Datos

Documentación funcional y técnica end-to-end

Repositorio `rag` · rama `agent-dev` · una entidad hipotecaria

Estado a 1 de septiembre de 2026 · Contexto regulatorio IFRS9

1. Qué es y qué problema resuelve

El Agente de Calidad de Datos evalúa reglas de calidad sobre las tablas del Data Warehouse de una entidad hipotecaria, y —sobre todo— **traduce reglas de negocio escritas en texto libre a reglas ejecutables**, usando un modelo de lenguaje local que **propone** y un Comité del dato que **revisa y aprueba** antes de que nada se ejecute contra datos reales.

PRINCIPIO DE DISEÑO

El agente **nunca activa una regla por su cuenta**. Toda propuesta pasa por una cola de revisión humana, y el despliegue final contra producción es un paso aparte, explícito y confirmado. El LLM es un asistente de traducción, no una autoridad.

El problema de fondo es que el catálogo de reglas de calidad está escrito por negocio en lenguaje natural («la fecha tiene que ser válida», «el campo no puede estar vacío», «tiene que tener 2 decimales»), mientras que el motor que las ejecuta necesita parámetros exactos. Traducir a mano ese catálogo es lento; traducirlo automáticamente es rápido pero arriesgado. El proyecto existe para hacerlo automáticamente *sin* asumir ese riesgo.

Proyecto hermano

Es un proyecto independiente del agente de documentación del diccionario (rama `main`, puerto 5000). Corre en su propio servicio Flask (puerto 5001), en la misma máquina que Ollama, y no comparte ciclo de despliegue con la aplicación de producción del diccionario. Solo reutiliza en modo **lectura** el sistema RAG del diccionario como contexto de negocio.

2. Topología: dónde vive cada cosa

Entender el proyecto exige entender que toca **tres sistemas distintos**, con permisos y niveles de riesgo muy diferentes.

Sistema	Ubicación	Rol y nivel de riesgo
Datos a analizar	SERVIDOR_DATOS\INSTANCIA DW_PRINCIPAL , DW_CRUDO , DW_LAGO	Solo lectura. Las tablas de hechos reales. Algunas reglas declaran DW_PRINCIPAL pero la tabla vive realmente en DW_CRUDO (origen AS/400), por lo que el código prueba las tres antes de rendirse.
Catálogo y motor	APP_CATALOGO APP_VALIDADORES	Lectura del catálogo de reglas; ejecución de los validadores. Son procedimientos de producción ajenos al proyecto , que pueden cambiar sin avisar.
Workspace propio	SERVIDOR_WORKSPACE WORKSPACE_AGENTE	Lectura y escritura. Todo lo que el agente produce (propuestas, bloques aprobados, resultados de pruebas) vive aquí, nunca en producción.
Modelo de lenguaje	localhost:11434 phi4-calidad	Ollama en local, en la misma máquina que el servicio. Ningún dato sale de la red interna. Es una variante propia de phi4:14b-q8_0 con la ventana de contexto ampliada (ver §8); el modelo base lo comparte el agente del diccionario y no se toca.

ADVERTENCIA PERMANENTE SOBRE EL MOTOR

Los validadores y el intérprete del mini-lenguaje de parámetros **no son de este repositorio**: son procedimientos SQL de producción escritos por otro equipo. Se han encontrado varias veces comportamientos reales que ni los manuales oficiales ni los comentarios del propio procedimiento documentaban. Lo que el proyecto sabe del motor es una **copia fechada de una auditoría**, no un enlace vivo. **Si algo no cuadra entre lo que dice el prompt y lo que hace el motor, hay que releer el procedimiento real antes de asumir que el LLM se equivocó.**

3. Arquitectura del repositorio

El código está organizado en paquetes Python por capa de arquitectura. Cada script se invoca como módulo desde la raíz (python -m paquete.modulo).

Paquete	Responsabilidad
nucleo/	Motor determinista de reglas y anomalías, más el orquestador. Evalúa una tabla contra un fichero de configuración y genera un informe. No usa LLM.
origen_reglas/	De texto libre a propuesta: detección de reglas activas, traducción vía LLM, registro en la cola de revisión, y el arnés de evaluación.
activacion/	De propuesta aprobada a algo ejecutable: generación del bloque T-SQL, o de la sentencia del mini-lenguaje (JQL), y su despliegue.
web/	Servicio Flask propio (puerto 5001): rutas /api/calidad/..., las tres páginas HTML y el <i>poller</i> que detecta reglas nuevas cada 300 s.
diccionario/	Dependencias de solo lectura sobre el sistema RAG del diccionario, que aporta contexto de negocio al LLM.
exploracion/	Herramientas sueltas de investigación contra el servidor real. No forma parte del flujo de producción.
_archivo/	Pipeline antiguo (SharePoint/Excel), archivado. No se toca.

4. El flujo end-to-end

- 1 **Detección** — Se leen las reglas vigentes del sistema nativo y se cruzan con los atributos físicos. Fuente autoritativa; la clave física ya viene resuelta.
- 2 **Traducción** — El LLM elige uno de tres mecanismos y produce los parámetros. Se le da el esquema real y la clave primaria real para que no invente.
- 3 **Verificación automática** — La propuesta se prueba y se somete a una batería de guardas. Aquí se cortan las traducciones que «funcionan» pero no miden lo que deben.
- 4 **Cola de revisión humana** — Queda PENDIENTE hasta que una persona la apruebe, corrija o rechace. Es el punto de control obligatorio del Comité del dato.
- 5 **Activación** — Al aprobar se persiste el artefacto en el workspace propio. Desplegarlo en producción sigue siendo un paso aparte y explícito.

4.1 Detección

Se leen de `APP_CATALOGO` las reglas con `Tb_Calidad_Regla.Es_Activa = 1`, cruzadas con `Tb_Calidad_Tabla_Atributo`. De ese cruce sale, por cada combinación regla + atributo físico, la **clave física ya resuelta** (`Clave_Fisico`), con la forma `Servidor.BaseDatos.Eschema.Tabla.Columna`.

Existen dos modos: **sincronización completa** (detecta y traduce todo de golpe) y **detección barata** sin LLM, que es lo que ejecuta el *poller* en segundo plano cada 300 segundos y deja los hallazgos en estado `NUEVA` para que un humano decida si merece la pena gastar una llamada al modelo.

4.2 Traducción con el LLM

Es el paso caro: cada llamada tarda entre 1 y 4 minutos. Antes de invocar al modelo, el sistema le prepara el contexto:

- **Esquema real de la tabla**, con precisión, escala y longitud reales — no solo el nombre del tipo. Sin ese dato el modelo no puede saber, por ejemplo, que una columna ya es `numeric(13,2)` y que por tanto exigirle 2 decimales es una comprobación que nunca puede fallar.
- **Clave primaria real**, buscada en las tres bases de datos candidatas, con respaldo en un índice único si no hay restricción formal.
- **Ejemplos confirmados** por revisores humanos como correctos, que se reutilizan como ejemplos de refuerzo.

El problema de las tablas anchas

Con tablas de muchas columnas el modelo devolvía un JSON válido pero con un **formato inventado**, sin llegar siquiera a agotar el tiempo de espera: no era un corte a mitad de respuesta, era perder de vista el formato de salida con un prompt demasiado grande. Calibrado contra una tabla de 86 columnas, el umbral quedó en **50 columnas**.

Por encima de ese umbral se hace un paso previo, más simple, que reduce el esquema a las columnas relevantes antes de traducir. Si esa llamada también falla, hay un **respaldo sin IA** por coincidencia de palabras. Y si aun así el prompt sigue siendo grande, la propuesta lleva un aviso para que el revisor la mire con más atención.

4.3 Los tres mecanismos de traducción

El modelo elige, **en este orden de preferencia**, cómo expresar la regla. La lógica es sencilla: cuanto menos SQL escrito a mano, menos hay que revisar y menos puede inventarse.

Mecanismo 1 — JQL (preferido)

Un mini-lenguaje de parámetros que el motor nativo ya interpreta, sin escribir SQL en absoluto. Tiene la forma

TIPO Etiqueta:valor Etiqueta:valor ...:

```
NUMERO Nulos:S Min:0 Dec:2 Clave:Id_Expediente,Id_Pais
FECHA Nulos:N EnDimTiempo:S Clave:Id_Operacion,Id_Pais
TEXTO Nulos:S Recorte:S MinLen:5 MaxLen:5 Clave:Id_Pais Filtro:Id_Pais=1
```

Tipos disponibles: NUMERO, TEXTO, FECHA, BOOLEANO, LISTA, UNICO.

LOS VALORES POR DEFECTO DEL MOTOR NO SON NEUTROS

Omitir una etiqueta no desactiva la comprobación: la deja en su valor por defecto, que a menudo *sí* hace algo. `Nulos:S` significa «no compruebo completitud»; `EnDimTiempo:S` valida además contra la dimensión tiempo; `MinDate:19890101` es el inicio de actividad de la entidad, no «sin límite». Por eso el sistema los emite siempre explícitos.

La gran ventaja de JQL es que se puede **probar de verdad contra datos reales sin publicar nada**: existe una variante del procedimiento del motor que recibe la sentencia en línea y, si se le pasa un identificador nulo, ni siquiera escribe en el log. Es la vía «medir primero, publicar después».

Mecanismo 2 — Validador genérico

Llama a un procedimiento que ya existe en producción (`Sp_Calidad_Numero`, `Texto`, `Fecha`, `Lista`, `Booleano`, `Unico`), volcando antes las filas a una tabla temporal. Es la salida cuando JQL no encaja o cuando tiene un fallo conocido.

Mecanismo 3 — Condición SQL personalizada

Último recurso: un fragmento SQL que describe las filas que **incumplen**, incrustado en un bloque generado. Es el más flexible y el más peligroso, porque obliga al modelo a escribir SQL sobre un esquema que solo conoce parcialmente.

4.4 Cola de revisión humana

Cada propuesta se registra en la tabla `CAL_Propuestas` del workspace propio, con la descripción original, la regla estructurada que propuso el LLM, su confianza y cualquier aviso generado. Los estados son `NUEVA` (detectada, aún sin traducir), `PENDIENTE` (traducida, esperando revisión), y `APROBADA` o `RECHAZADA`.

El revisor puede además **corregir** la propuesta —editar el JQL a mano, forzar la clave primaria cuando la detección automática no la encontró— y regenerar el artefacto **sin volver a llamar al LLM**.

4.5 Activación y despliegue

Al aprobar, el artefacto se persiste según el mecanismo, en dos tablas **deliberadamente separadas**: los mecanismos que generan SQL van a `CAL_Bloques_Sp_AgenteIA`, mientras que JQL va a `CAL_Bloques_JQL`, porque no genera ningún bloque T-SQL y forzarle esas columnas no tendría sentido.

LA FRONTERA CON PRODUCCIÓN

Persistir el artefacto en el workspace **no es desplegarlo**. El despliegue real —ensamblar el procedimiento completo y ejecutar un `CREATE OR ALTER PROCEDURE` contra `DW_PRINCIPAL`— es una función aparte que **por defecto no ejecuta nada**: devuelve el SQL para que se revise, y solo lo aplica si se le pide explícitamente. El procedimiento generado nace además con el modo simulacro activado por defecto.

5. El arnés de evaluación: cómo se sabe si el agente funciona

Esta es la pieza que convierte el proyecto en algo medible en lugar de una impresión. Compara, en segundo plano, cómo traduce el LLM reglas **ya implementadas a mano** contra el bloque real que escribió el equipo.

5.1 Mecánica

- Se lanza un **lote** con dos listas: reglas ya implementadas (comparables contra su bloque real) y reglas activas todavía sin implementar (sin comparación posible, solo para revisión).
- Cada regla se reintenta hasta un techo configurable (por defecto 20), parando antes si acierta dentro de los primeros 5.
- Cada resultado se guarda según va terminando, para poder ver resultados parciales sin esperar a que acabe el lote entero —que puede durar horas.
- Se registran tiempo y **tokens de entrada y salida** por intento, para diagnosticar coste y riesgo de contexto con datos, no con intuición.

5.2 Los dos veredictos, y por qué hacen falta los dos

Cada fila tiene **dos juicios independientes**:

Veredicto	Qué es
Coincide (automático)	Heurística de comparación contra el bloque real. No es equivalencia semántica SQL . A menudo no puede decidir y queda sin valor.
Correcta_Humano (manual)	Juicio de un revisor, con notas libres. Es el que de verdad importa.

Comparar ambos campos **expone dónde falla el propio arnés**: si el humano dice «correcta» y la heurística dice «no coincide», es un falso negativo del arnés, no un fallo del agente. Las filas confirmadas correctas se reutilizan como ejemplos de refuerzo para el prompt.

POR QUÉ ESTO RESULTÓ SER LA PIEZA CLAVE

El etiquetado humano no solo mide: es lo que permitió **validar cada guarda nueva antes de darla por buena**, exigiendo que no bloqueara ninguna traducción que un revisor hubiera dado por correcta. Ese contraste descartó una guarda que parecía evidente y era falsa (§7.3).

6. Lo que se ha aprendido del motor real

Auditar el código real de los validadores —no solo su documentación— destapó comportamientos que ningún manual recogía. Están todos codificados como validaciones activas.

Hallazgo	Efecto real	Mitigación
Fallo de <code>@AdmiteNull</code>	En las ramas TEXTO y LISTA, el parámetro que se pasa al validador es el de <i>otra</i> rama, por un copia-pegar. Nunca se asigna, así que llega nulo: un valor nulo jamás se marca inválido por mucho que se pida.	Se prohíbe TEXTO con comprobación de nulos; se redirige al validador genérico, que llama al procedimiento directamente sin pasar por el fallo.
BOOLEANO no valida	No marca inválidos los valores fuera de lista: los normaliza a 0/1 sin avisar. Una regla de «valores válidos» mediría 100 % de cumplimiento siempre .	BOOLEANO ni siquiera se ofrece como opción al modelo; se usa LISTA.
<code>MaxDate:99991231</code> es dinámico	El motor lo traduce al 31/12 del año siguiente, así que un artefacto con ese literal mide un rango distinto cada año .	Prohibido. Para no acotar por arriba, se omite la etiqueta.
<code>Dec:</code> sí existe	La precisión decimal <i>sí</i> está implementada y conectada, pese a no aparecer en ningún manual ni en los ejemplos del propio procedimiento.	Se documentó y se usa. Descubierto releendo el procedimiento tras una intuición del usuario que contradecía lo que se creía.

7. El problema central: el «100 % de cumplimiento falso»

El hallazgo más importante del proyecto hasta la fecha. Al revisar un lote completo con todas las filas ya etiquetadas apareció un patrón que se repetía con **cuatro caras distintas**:

EL PATRÓN

Una regla se traduce a algo **formalmente correcto**. Pasa la validación gramatical. Se ejecuta sin ningún error. No marca ni una sola fila. Y por tanto informa de un **100 % de cumplimiento que no es real**, porque no comprueba lo que la regla pedía.

En un contexto regulatorio esto es **peor que un fallo ruidoso**: un error visible se corrige; una garantía de calidad que nadie ha medido se da por buena.

7.1 Las cuatro caras

Caso	Por qué pasaba desapercibido
Regla de decimales sin la etiqueta que los mide	Omitir esa etiqueta es <i>neutro</i> en el motor. Una regla «tiene que ser un número entero» sin ella no comprueba nada sobre los decimales.
Clave de trazabilidad inexistente	El motor acepta sin quejarse una clave que no es una columna real, porque solo la usa para identificar filas del resultado. El modelo llegó a copiar el nombre descriptivo del catálogo como si fuera una columna.
Regla sin ninguna comprobación dentro	Reglas de completitud que acababan con la comprobación de nulos desactivada y sin ningún otro chequeo: no pueden marcar una fila mire los datos que mire.
Tabla maestra inventada	El motor de base de datos resuelve los nombres en diferido: el bloque se genera y hasta se despliega sin queja, y falla en la primera ejecución real en producción .

7.2 La lección: el prompt no basta

EVIDENCIA DIRECTA, MEDIDA EN EL PROPIO PROYECTO

El prompt *ya* prohibía inventar tablas maestras y pedía, si no se sabe el nombre real, usar SQL personalizado y bajar la confianza. En una regla concreta el modelo **hizo exactamente las dos cosas que se le pedían...** y aun así escribió el nombre de una tabla inexistente.

Se reforzó después el prompt para otro caso, con instrucciones explícitas y en mayúsculas. Al relanzar la evaluación, las tres reglas afectadas **volvieron a fallar igual**. Lo único que las paró fue la verificación en código.

Conclusión operativa: cada instrucción del prompt que de verdad importe necesita además una comprobación automática que la verifique contra el esquema real o contra el propio texto de la regla.

7.3 El método de validación de guardas

Toda comprobación nueva se contrasta contra las etiquetas humanas del lote completo antes de darla por buena. El criterio de aceptación es **cero casos de «bloquea algo que el revisor dio por correcto»**.

No es teórico: ese contraste tumbó una guarda que parecía evidente —dar por hecho que «el campo no puede estar vacío» obliga siempre a comprobar nulos—. Habría bloqueado dos traducciones válidas, porque **en estas tablas «vacío» no siempre significa nulo**. Quedó documentada en el código para que nadie la reintente.

Corolario útil: **las guardas estructurales** —las que solo miran la sentencia generada— **aguantan mejor que las basadas en el texto en español**, porque el catálogo redacta lo mismo de muchas formas y el significado depende de la columna.

7.4 Resultado medido

Contraste sobre las 62 traducciones JQL del lote de referencia, usando las etiquetas humanas como patrón: **21 filas bloqueadas, todas marcadas incorrectas por el revisor; 0 regresiones** sobre las 21 que dio por buenas. En una verificación posterior con el código ya desplegado, sobre 9 reglas previamente defectuosas: 7 bloqueadas con mensaje accionable, 1 abstención del modelo, 1 mejorada. **Ninguna sigue produciendo un cumplimiento falso.**

8. La ventana de contexto: un fallo invisible durante meses

Descubierto el 1 de septiembre de 2026, y merece sección propia porque su forma es distinta a todo lo anterior: no era un error en el código de este proyecto, sino una **configuración por defecto** de la herramienta que sirve al modelo.

8.1 El síntoma

Investigando cómo reducir el tamaño de lo que se le envía al modelo, se midió el consumo real por llamada. Y no variaba: de 113 intentos registrados, **64 daban exactamente el mismo número**, y ninguno lo superaba nunca. Un 57% de los casos apilados sobre un único valor no es una distribución natural — es un techo.

EL DIAGNÓSTICO

El modelo admite de forma nativa **16.384** tokens de contexto. Nada en el sistema lo configuraba, así que la herramienta usaba su valor por defecto: **4.096**. Y los prompts necesitan de verdad entre **4.800 y 5.400**.

Resultado: en más de la mitad de las traducciones se descartaba en silencio **entre un 13% y un 24% del enunciado**. El motor no da error: simplemente corta.

8.2 Lo que explica hacia atrás

Dos comportamientos que se venían arrastrando sin entender quedan explicados:

- **Que el modelo «perdiera el formato de salida» en tablas anchas.** Se atribuía a que un prompt largo lo despistaba. La causa real es más simple: las instrucciones de formato, que van al final, eran justamente las que se cortaban.
- **Que reforzar el prompt no sirviera de nada.** El 26 de agosto se añadieron instrucciones explícitas para un caso concreto y al repetir la evaluación no cambió absolutamente nada. El texto añadido caía fuera de la ventana: **nunca llegó a leerse**.

8.3 La corrección y su verificación

Se creó una variante propia del modelo con la ventana al doble. No implica reentrenar ni duplicar nada: son los mismos ficheros de pesos con un parámetro distinto. Se mantiene separada del modelo original porque lo comparte el agente del diccionario, que convive en la misma máquina.

Medición	Antes	Después
Tokens de entrada	tope en 4.095	4.787 – 5.352
Llamadas contra el techo	64 de 113	0
Reglas de fecha con mecanismo nativo	0 de 3	3 de 3
Reglas decimales con su parámetro	0 de 3	3 de 3

LA LECCIÓN, QUE CORRIGE A LA ANTERIOR

En la sección 7 se concluyó que «el prompt no basta». Este hallazgo la matiza: el prompt **sí** estaba bien escrito — no llegaba entero. Ambas cosas son ciertas y no se contradicen: las verificaciones en código siguen siendo necesarias, pero **antes de dar por inútil una instrucción, hay que comprobar que el modelo la está recibiendo**.

El dato que lo delató llevaba meses guardado en la tabla de resultados. Los dos indicadores que el sistema vigilaba —número de columnas y longitud del texto— eran aproximaciones, y ninguna lo detectó. El aviso se calcula ahora con el consumo real de cada llamada.

9. Operación

9.1 Arranque local

```
pip install -r requirements.txt
python -m web.servir_calidad          # sirve en http://localhost:5001
```

Páginas: `/calidad` (informe del motor determinista), `/calidad/reglas` (cola de revisión) y `/calidad/evaluacion` (banco de pruebas con etiquetado).

9.2 Despliegue

Corre como servicio de Windows (`ServicioAgenteCalidad`, vía NSSM) en `servidor-workspace.interno`, bajo una cuenta de dominio con acceso a los tres saltos de SQL Server. El árbol desplegado vive en `C:\ruta\despliegue`. No hay script de despliegue en el repositorio: se sincronizan los ficheros versionados y se reinicia el servicio.

10. Estado actual y limitaciones conocidas

10.1 Limitaciones que no son fallos del código

Del análisis del lote completo, buena parte de las traducciones que un revisor marca como incorrectas **no son corregibles por software**, y conviene tenerlo claro:

- **Filtros de población que el agente no puede conocer.** El propio revisor lo anota: «aunque el agente no tiene cómo saberlo». La información no existe en ningún sitio consultable. Solución real: documentar el filtro en el catálogo.
- **Claves de negocio no declaradas.** Tablas crudas sin clave primaria ni índice único, cuya clave real es una combinación de expresiones que solo conoce una persona.
- **Reglas de negocio complejas.** Fórmulas que combinan varias columnas quedan fuera del alcance de los mecanismos nativos por diseño.

10.2 Pendiente

- **Publicar JQL en el catálogo** para que lo ejecute el motor de producción, no solo la prueba en línea. Decisión deliberada, aplazada.
- **Trazabilidad solicitada por el equipo:** persistir la consulta o procedimiento ejecutado y la población seleccionada, para poder replicar manualmente una ejecución. Pendiente de confirmar si las tablas actuales ya lo cubren.
- **Rangos inventados por el modelo:** sigue proponiendo límites numéricos que no aparecen en el texto de la regla. Hoy quedan interceptados por otras guardas, pero el comportamiento persiste.

RESUMEN EN UNA FRASE

Un sistema que traduce automáticamente reglas de calidad de negocio a reglas ejecutables, con un modelo local que propone y un humano que aprueba, y cuya lección principal es que **la seguridad no está en lo que se le pide al modelo, sino en lo que se verifica de su respuesta.**

